Swarm has a list of remote APIs similar to Docker. This enables all kinds of Docker clients to connect to Swarm. In addition, the Swarm API provides the cluster information as well.

Docker API information is restricted to one single Docker engine, whereas the Docker Swarm API provides the information on the cluster of engines, the number of nodes available in a cluster, and the node details.

Raft Consensus

Manager nodes use the Raft Consensus Algorithm to internally manage the cluster state.

This is to ensure that all manager nodes that are scheduling and controlling tasks in the cluster maintain/store a consistent state.

Docker Swarm: Commands

Let us now get acquainted with Docker Swarm commands.

To initialise Swarm, use the following command:

```
docker swarm init ↩
```

Note: You can configure the manager node to publish its address as a manager with the mentioned IP address, as shown below:

```
docker swarm init --advertise-addr <ip-address>
```

For Docker Swarm Help, use the following command:

```
docker swarm --help ↩
```

The above command lists the basic Swarm commands with their usage details.

To list the number of nodes currently available in the Swarm, use the following command:

```
docker node ls ↩
```

Deploying a service with constraints

You can add your own constraints while deploying a service. This will ensure that the service will get scheduled as a task only when the constraint is met. This constraint is known as placement.

In the following example, you add a constraint to deploy a service only on nodes where there is SSD storage.

```
docker service create --name testService --replicas 2
--constraint node.labels.disk==ssd tomcat
```

By: Neetesh Mehrotra

The author works at NIIT Technologies as a senior test engineer. His areas of interest are Java development and automation testing. He can be contacted at *mehrotra.neetesh@gmail.com*.

Continued form page...59

Mapping storage: The application's data is stored inside a container and, hence, when we destroy the container, data is also deleted. To avoid this, we can map volumes from the container to a local directory on a host machine. We can achieve this by using the *-v* option, as follows:

```
# docker run -v /opt/mysql-data:/var/lib/mysql mysql
```

In the above example, we have mapped the container's */var/lib/mysql* volume to the local directory */opt/mysql-data*. Because of this, data can be persisted even when the container is destroyed.

Mapping ports and volumes with Docker Compose: To map ports and volumes as a part of the Docker Compose process, add the attributes of the ports and volumes in the 'docker-compose.yaml' file. After doing this, our modified file will look like what follows:

```
version: '2.0'

services:
  database:
    image: "jarvis/acme-web-app"
    ports: ---------------------------->
        - "80:5000" ---------------------------->
    web:
      image: "mysql"
      volumes: ---------------------------->
            - "/opt/mysql-data:/var/lib/mysql" ----------
------------------>
```

Docker cluster: So far, we have worked with a single Docker host. This is a bare-minimal setup and is good enough for development and testing purposes. However, this is not enough for production, because if the Docker host goes down, then the entire application will go offline. To overcome this single point of failure, we can provide high availability to the container by using a Swarm cluster

A Swarm cluster is created with the aid of multiple Docker hosts. In this cluster, we can designate one of the nodes as a master and the remaining nodes as workers. The master will be responsible for load distribution and providing high availability within the cluster, whereas workers will host the Docker container after co-ordinating with the master.

In this article, we have discussed the basics of Docker as well as touched upon some advanced concepts. The article is a good starting point for absolute beginners. Once you build a strong foundation in Docker, you can delve deep into individual topics that interest you.

By: Narendra K.

The author is a FOSS enthusiast. He can be reached at *narendra0002017@gmail.com*.